



YOJNA SETU — AGENT VISUALIZATION SPEC

Chakravyuha: Live Agent-Run Animation

An on-demand 3D visualization, triggered only when agents are actively running (Sathi, Sahayak Lens, and other agent-status surfaces) — built to sit alongside, not conflict with, the existing background motion system.

1. Concept

WHY THE CHAKRAVYUHA

Core idea

The spiral formation as a processing pipeline

The Chakravyuha is a layered, spiral military formation from the Mahabharata — a task/request enters from the outside and must pass through successive layers to reach the center. This maps naturally onto a 12-agent pipeline: each layer of the spiral is a stage of processing, and watching a request travel inward visually shows the user which agents are handling their task, in what order, right now.

2. When It Appears (Trigger Rules)

THIS IS A RUN-STATE VISUALIZATION, NOT AMBIENT MOTION

Condition	Behavior
No agent task running	Chakravyuha is fully hidden / unmounted. Existing background motion continues undisturbed.
User triggers an agent action (Sathi query, Sahayak Lens scan, any pipeline run)	Chakravyuha mounts and renders <i>only within that feature's status panel</i> — not full-screen, not layered over background motion.
Task completes or errors	Brief resolve animation (light reaches center / fades on error), then unmounts after a short delay.
Multiple agent runs at once	Multiple light-points enter the same spiral from different outer points, each tagged to its own task, converging independently.

Isolation from background motion: Mount the Chakravyuha canvas in its own dedicated container inside the agent-status component (Sathi panel / Sahayak Lens panel), with its own Three.js scene and renderer instance — do not share the scene graph or renderer with the background motion system. This avoids z-index conflicts, competing camera controls, and frame-rate contention. Pause or reduce the background motion's render loop (not remove it) while this panel is active, if both run in view at once, to protect performance.

3. Visual Structure

HOW THE SPIRAL IS BUILT

Geometry

Concentric spiral rings viewed at a tilted angle (not flat top-down) so depth is visible — near rings occlude far rings as the camera holds a slight orbit. Rings narrow toward a glowing center point.

Agent placement

Each of the 12 agents occupies a fixed gate/node position along the spiral, ordered by pipeline stage. Idle agents render as dim, low-opacity nodes; visually present at all times the panel is open, but inactive until reached.

Task traversal

The active task is a single point of light entering from the outer ring. As it reaches each agent's gate, that node brightens and pulses for the duration of that agent's processing, then dims as the light moves to the next layer. The light leaves a short fading trail so users can see the path already taken.

Resolution

On success, the light reaches the center and the whole spiral gives a soft unified glow pulse before the panel resolves to the normal result view. On error, the light dims/stops at the failing agent's node rather than reaching the center — giving users an honest visual of where things stopped.

4. Tone Note on the Source Myth

Keep the structure, not the tragedy: In the Mahabharata, Abhimanyu enters the Chakravyuha and dies inside it, unable to breach the final layer. For a platform meant to reassure citizens their application is safely progressing, only the formation's *structure* (layered spiral, sequential traversal) should be referenced — not the fatal outcome of the story. Avoid any framing, copy, or animation beat that implies the task could get "trapped" or "lost" inside — every run should visually resolve to either success at the center or a clear, honest stop-point on error, never an ambiguous or ominous dead-end.

5. Performance Notes

SINCE THIS MOUNTS ON TOP OF AN ALREADY-RUNNING BACKGROUND SCENE

Concern	Approach
---------	----------

Two 3D scenes at once	Separate renderer/canvas per component; throttle background motion's frame rate while agent panel is active.
Low-end devices	Fallback to a 2D animated SVG spiral (Framer Motion) with the same traversal logic if WebGL/perf budget is constrained.
Mount/unmount cost	Lazy-init the Three.js scene on first agent run, then keep it warm (hidden, not destroyed) for subsequent runs in the same session to avoid re-init lag.
Reduced motion	Respect <code>prefers-reduced-motion</code> : replace traversal animation with a static stepped progress list.
✦	